CS/DS 541: Deep Learning **Fall 2026**

Homework 1

Due: 5:59pm ET Thursday September 10

This problem can be done in teams of up 2 students.

Problem 1: Understanding Numpy Broadcasting (6 points)

In linear algebra, the standard matrix summation of a matrix A by a matrix B (denoted as $A+B$) has strict dimensional requirements: the shapes of A and B must be the same.

However, numerical computing libraries like NumPy implement a powerful feature called **broadcasting**. Broadcasting allows for operations on arrays of different shapes, providing flexibility and enabling more concise code. The goal of this problem is to explore the rules and behaviors of broadcasting, and to understand the errors that can arise when array dimensions are not compatible.

Instructions

For each of the following scenarios, use `np.random.randn()` to generate the initial random arrays. Do not use `np.matrix`; all operations should be performed with `np.array`. For each case, provide your Python code and state the shape of the resulting array. The answers can be provided as a comment inside each section the code.

Questions

a) Sum (+) with a Column Vector (2 points)

- Create a 2D array `A` with a shape of `(4, 3)`.
- Create a 2D array `B` with a shape of `(3, 1)` using `np.array([[0],[1],[2]])`.
- Sum `A` and `B` using the `+` operator.
- **What is the shape of the resulting array?** *Hint: This will produce an error. Explain why the broadcasting fails in this case.*

b) Sum (+) with a Row Vector (2 points)

- Create a 2D array `A` with a shape of `(4, 3)`.
- Create a 2D array `C` with a shape of `(1, 3)` using `np.array([[0, 1, 2]])`.
- Sum `A` and `C` using the `+` operator.
- **What is the shape of the resulting array? How to interpret the result?**

c) Sum (+) with a 1D Array (2 points)

- Create a 2D array `A` with a shape of `(3, 3)`.
- Create a 1D array `D` using `np.array([0, 1, 2])`.
- Sum `A` by `D` using the `+` operator.
- **What is the shape of the resulting array? How does broadcast allows the sum between these arrays, given that they originally have different shapes?**

Problem 2: MLP Forward Propagation with NumPy (11 points) [2]

In this problem, you will implement the forward propagation step for a simple Multi-Layer Perceptron (MLP) with one hidden layer and a ReLU activation function. The goal is to understand the matrix operations involved and the conventions for handling data.

Instructions

For part (a) provide the required Python code. For parts (b) and (c) provide a written explanation.

a) Implementation of Forward Propagation Functions (5 points)

Define the following four Python functions. Each function should include a detailed docstring that describes its parameters, and the expected shapes for each argument. Describe it in terms of variables (e.g., n); do not choose specific values for for the dimensions. The activation function for the hidden layer is ReLU, defined as $\text{ReLU}(z) = \max(0, z)$. The output is a scalar $\hat{y} \in \mathbb{R}$.

- `ReLU(z)`: This function takes an input vector and, for each of its elements, returns the maximum between the element and zero.
- `forwardPropA(W1, b1, w2, b2, x)`: This function should compute the forward pass for a **single input vector** `x` using **left multiplication** convention for weights, i.e. the same used in our class).
- `forwardPropB(W1, b1, w2, b2, X)`: Extend `forwardPropA` to handle a **dataset** `X`. This function should also use left multiplication.
- `forwardPropC(W1, b1, w2, b2, x)`: This function should compute the forward pass for a **single input vector** `x` using **right multiplication** convention for weights.
- `forwardPropD(W1, b1, w2, b2, X)`: Extend `forwardPropC` to handle a **dataset** `X`. This function should also use right multiplication.

b) Applying the Functions to a Dataset (4 points)

Suppose you have a dataset `X` consisting of 100 observations, where each observation is a vector with 5 features. The hidden layer of your MLP has 10 neurons, and the output is a single value. Explain how you would set up the weight matrices, bias vectors, and the data matrix `X` to compute the MLP predictions for all 100 observations using either the left-multiplication functions (`forwardPropA` and `forwardPropB`) or[1] the right-multiplicatiom functions (`forwardPropC` and `forwardPropD`) you defined in part (a). Specify the shapes of all inputs (`W1`, `b1`, `w2`, `b2`, `X`/`x`).

c) Performance Argument (2 points)

Without necessarily running empirical tests, argue which of your two functions in part (b) should be faster for processing a large dataset.

Problem 3: Linear Regression via Gradient Descent (17 points)

In this assignment, you will implement a linear regression model from scratch using NumPy and train it using gradient descent. The goal is to predict age based on facial images[2]. Before coding the solution, you will need to derive an expression for the gradient of the weight vector.

Consider the linear regression model $\hat{y} = b + w_1 x_1 + \ldots + w_m x_m$. For simplicity in this problem, we will assume the bias term b is incorporated into the weight vector $\mathbf{w}$ by prepending a constant feature of 1 to each input vector $\mathbf{x}$. The model can then be written as $\hat{y} = \mathbf{x}^T \mathbf{w}$.

[1] Yes, this is an exclusive or. You only need to do two functions.

[2] This may have serious implications; happy to talk about it.

The Mean Squared Error (MSE) loss is given by $f_{\text{MSE}}(\mathbf{w}) = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}^{(i)} - y^{(i)})^2$. We saw in class that the derivative for the bias term is $\frac{\partial f_{\text{MSE}}}{\partial b} = \frac{2}{n}\sum_{i=1}^{n}(\hat{y}^{(i)} - y^{(i)})$.

Questions

a) Following a similar derivation, obtain the partial derivative of the MSE loss with respect to a single weight w_j, i.e., find $\frac{\partial f_{\text{MSE}}}{\partial w_j}$. Show your steps. (2 points)

b) Using your result from part (a), write down the expression to compute the full gradient, $\nabla_{\mathbf{w}} f_{\text{MSE}}$, in a vectorized form. The shape of the gradient must be the same as the shape of the weight vector $\mathbf{w}$. (2 points)

c) Train a linear regression that analyzes a grayscale face image and outputs a real number $\hat{y}$ that estimates how old the person is (in years). The training and testing data are available here: `https://canvas.wpi.edu/courses/87875/files/8763694/download?download_frd=1` The dataset consists of:

- Xtr.npy: Training images (images of shape 48×48)
- ytr.npy: Training labels (ages)
- Xte.npy: Testing images (images of shape 48×48)
- yte.npy: Testing labels (ages)

You will need to complete the code in the provided starter file `homework1_template.py`. The following functions need to be implemented or completed:

```
--------------------------------------------------------------------------------
Function: gradient_descent(x_train, y_train, w0, alpha, num_steps) [3 points]
--------------------------------------------------------------------------------
This function performs gradient descent to learn the weights of the linear model.

Tasks:
1. Initialize weights: [1 point]
   - Create a deep copy of the initial weights ‘w0‘ to avoid modifying the
   original array.

2. Iterate for ‘num_steps‘: [2 points]
   a. Compute Loss:
      - Calculate the predicted ‘y‘ values using the current weights ‘w‘ and
      the training data ‘x_train‘.
      - Compute the Mean Squared Error (MSE) loss between the predicted ‘y‘
      and the true ‘y_train‘.

   b. Compute Gradients and Update Parameters:
      - Calculate the gradient of the loss with respect to the weights ‘w‘.
      - Update the weights ‘w‘ using the learning rate ‘alpha‘ and the computed
      gradient.

--------------------------------------------------------------------------------
Function: train_linear_regression(x_train, y_train) [3 points]
--------------------------------------------------------------------------------
This function initializes the model and orchestrates the training process.

Tasks:
```

```
1. Initialize Weights:
   - Set a random seed for reproducibility.
   - Initialize the weights ‘w0‘ using a normal distribution. The standard
   deviation should be scaled by the square root of the number of features.

2. Initial Prediction and Loss:
   - Calculate the initial predictions ‘y_pred‘ using the initial weights ‘w0‘.
   - Compute and print the initial training loss (MSE).

3. Run Gradient Descent:
   - Call the ‘gradient_descent‘ function with the appropriate parameters to
   train the model.

--------------------------------------------------------------------------------
Function: main() [3 points]
--------------------------------------------------------------------------------
This is the main function where the script execution begins.

Tasks:
1. Load Data:
   - The data loading part is already provided.

2. (Optional) Pre-processing:
   a. Feature Standardization:
      - Calculate the mean and standard deviation of the *training* features ‘X_tr‘.
      - Standardize both the training (‘X_tr‘) and testing (‘X_te‘) sets
      using the mean and standard deviation calculated from the training set.

   b. Add Bias Term:
      - Prepend a column of ones to both ‘X_tr‘ and ‘X_te‘. This allows the bias term
        to be treated as a regular weight.

3. Train Model:
   - Call ‘train_linear_regression‘ to get the trained weights ‘w‘.

4. Evaluate Model:
   - Calculate and print the final Mean Squared Error (MSE) on both the
     training data and the testing data separately.
```

Run it for a predetermined number of epochs $T = 100$. An *epoch* is defined as a completing passing over the data.

Note: you must complete this problem using only linear algebraic operations in `numpy` – you may **not** use any off-the-shelf linear regression software, as that would defeat the purpose.

Find the optimal weights $\mathbf{w} = (b, w_1, \ldots, w_{2304})$ for the linear regression model via gradient descent using the expression you derived for the gradient of the cost function in part (b) w.r.t. $\mathbf{w}$.

After optimizing $\mathbf{w}$ only on the **training set**, compute and report the cost f_{MSE} on the training set $\mathcal{D}_{\text{tr}}$ and (separately) on the testing set $\mathcal{D}_{\text{te}}$. Please report these numbers in the PDF file. (With an appropriately chosen learning rate, you can get MSE < 1300). (9 points)

d) Show how to change your code above to stop once a stopping criterion has been satisfied. Think about a good stop criterion and explain your choice. (2 points)

e) Now run two additional rounds of training, one with a larger and the other with a smaller learning rate than the one you used in part (c). Plot the three loss curves and identify which learning rates was more effective. (2 points)

Problem 4: The Necessity of Non-Linear Activations (10 points)

A key design choice in Feed-forward Neural Networks is the use of non-linear activation functions (like ReLU, sigmoid, tanh) between layers. In this problem, you will prove that without these non-linearities, a deep network of linear layers is mathematically equivalent to a single linear layer, thus losing the ability to model complex patterns.

Derivation Steps

Consider a 3-layer neural network without any activation functions. The input layer is a vector $\mathbf{x}$, the first hidden layer has weights $\mathbf{W}^{[1]}$ and bias $\mathbf{b}^{[1]}$, and the output layer has weights $\mathbf{W}^{[2]}$ and bias $\mathbf{b}^{[2]}$.

a) Write down the mathematical expression for the output of the first linear layer. Let's call this output $\mathbf{z}^{[1]}$ (2 point).

b) Now, write down the expression for the output of the second linear layer, which we'll call $\hat{\mathbf{y}}$. The input to this layer is the output from the first layer, $\mathbf{z}^{[1]}$ (2 point).

c) Substitute the expression for $\mathbf{z}^{[1]}$ from part (a) into your equation from part (b). Expand the resulting expression algebraically (2 point).

d) Show that the expression from part (c) can be rewritten as a single linear transformation of the form $\hat{\mathbf{y}} = \mathbf{W}'\mathbf{x} + \mathbf{b}'$. To do this, you must define what the equivalent single weight matrix $\mathbf{W}'$ and equivalent single bias vector $\mathbf{b}'$ are in terms of the original parameters ($\mathbf{W}^{[1]}, \mathbf{b}^{[1]}, \mathbf{W}^{[2]}, \mathbf{b}^{[2]}$) (2 points).

e) Based on your result in part (d), explain in one or two sentences why this demonstrates that non-linear activation functions are essential for giving neural networks their representational power (2 points).

Problem 4: Approximating a Function with Pulses (6 points)

We saw in class the intuition behind the universal approximation theorem for MLPs with one hidden layer. By using pairs of sigmoid neurons, we can create "pulse" functions. A single pulse can be defined by three parameters: the start of the pulse (s_1), the end of the pulse (s_2), and the height of the pulse (h). By summing many of these pulses, we can approximate any continuous function.

Learning Goal

Understand how the pulses were constructed in the "intuitive, non-formal proof" of the universal approximation theorem that we saw in class.

Task

Your goal is to approximate the function $f(x) = x(4-x)$ on the interval $[0, 4]$ using exactly four pulses. The function is plotted below for your reference.

Choose the appropriate triples (s_1, s_2, h) for four pulses that, when summed together, create a reasonable approximation of the function $f(x)$. You can define your pulses by dividing the domain $[0, 4]$ into four equal segments. For each segment, determine the start, end, and an appropriate height for the pulse.

(4 points) Provide your answer as a list of four triples. For example:

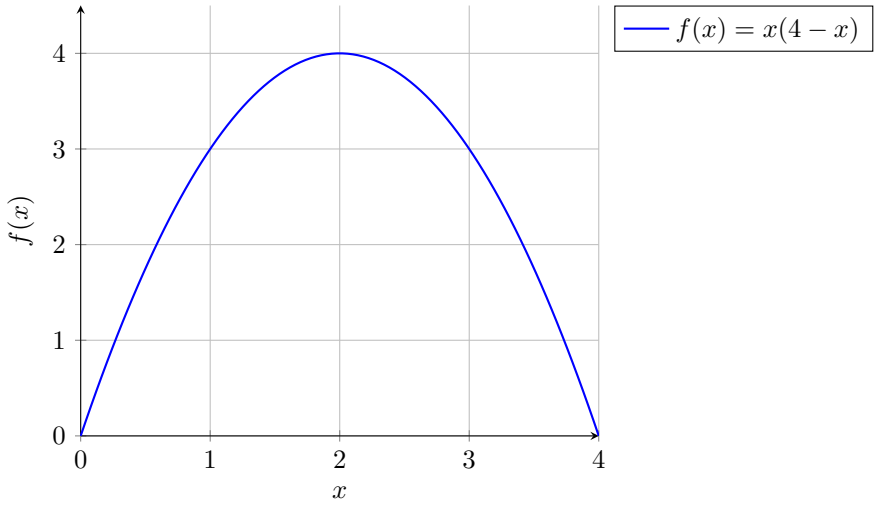

Figure 1: The target function to be approximated.

- Pulse 1: $(s_1, s_2, h) = (..., ..., ...)$
- Pulse 2: $(s_1, s_2, h) = (..., ..., ...)$
- Pulse 3: $(s_1, s_2, h) = (..., ..., ...)$
- Pulse 4: $(s_1, s_2, h) = (..., ..., ...)$

(2 points) Now draw a multi-layer perceptron with 1 hidden layer that represents an approximation to these pulses. Indicate your choices for all weights and biases.

Problem 5: Stochastic Gradient Descent (10 points)

First, consider the case where the mini-batch consists of a single observation $\widetilde{n} = 1$ sampled uniformly at random **with replacement**.

1. In this case, we will define an epoch as a sequence of n optimization iterations. What is the probability p_{never} that one particular observation is never sampled during one epoch? (Write it as a function of n). [2 points].

2. Using the fact that $\lim_{n\to\infty}(1 + 1/x)^x = e$, prove that, for large x, $p_{\text{never}} \approx 0.3679$. [2 points].

Now, let's go back to the typical case where a minibatch has size $\widetilde{n} > 1$. For simplicity, we will continue to assume sampling with replacement.
A number of recent empirical results show that the right amount of gradient variance in SGD is essential to train some types of deep neural networks. In particular, (Goyal et al., NeurIPS 2017) trains ResNet-50 on ImageNet in 1 hour by adjusting the learning rates as a function of minibatch size to keep the gradient noise constant. The authors argue: *"When the minibatch size is multiplied by k, multiply the learning rate by k."*

1. Let the variance of the gradient computed over one random observation be $\text{Var}(\nabla f(\mathbf{x}^{(i)}, y^i)) = \sigma^2$. What is the variance of the gradient computed over a minibatch? (Note: In reality, the variance should be a vector with the same dimension as the number parameters, but we will keep it simple) [2 points].

2. Considering that the minibatch gradient is an average of i.i.d. random variables, what happens to the variance when you multiply the minibatch size by k? [2 points].

3. Why is the claim in the paper incorrect? How can you change the learning rate so that the gradient noise stays constant? [2 points].

Submission

Create a Zip file containing both your Python and PDF files, and then submit on Canvas. If you are working as part of a group, then only **one** member of your group should submit.

Teamwork

You may complete this homework assignment either individually or in teams up to 2 people.